

Leo Rover Navigation Stack — Technical Brief for Claude Code

Verbatim terminal outputs, error messages, and current state of an unresolved map → odom TF problem in a ROS 2 Jazzy multi-machine SLAM setup

Audience	Claude Code (autonomous coding agent)
Purpose	Use this brief to understand current system state and propose / apply code changes
Critical instruction	Do NOT repeat fixes already listed in Section 5; they are already applied
Top-level goal	Get map → odom TF publishing so Nav2 + RViz demo works
Hardware	Leo Rover + RPLidar A2M12 + NUC + Raspberry Pi
ROS distribution	Jazzy Jalisco (NUC: Ubuntu 24.04, Pi: LeoOS 2.3.0 / Ubuntu 24.04.3)
Workspace	/home/team7/leo_test (non-standard layout, package files at root)
Compiled	27 April 2026

NOTE: All terminal outputs in this document are reproduced verbatim from the user's session. Section 6 contains the most recent (and currently unresolved) terminal output. Section 9 contains specific code-change guidance for Claude Code.

Contents

- 1. System Topology & File Locations
- 2. The Single Unresolved Problem
- 3. Confirmed Working Subsystems
- 4. Current Configuration Files (relevant excerpts)
- 5. Already-Applied Fixes (DO NOT REPEAT)
- 6. Verbatim Terminal Outputs From Latest Session
- 7. SLAM Toolbox Behaviour Summary
- 8. Probability Table for Remaining Root Cause
- 9. Specific Tasks for Claude Code
- 10. Reference: Diagnostic Commands & Expected Outputs

1. System Topology & File Locations

Network

Host	Identity	IP	Role
NUC	team7-NUC12WSKi7	10.0.0.35	Ubuntu 24.04 Noble, ROS 2 Jazzy, runs SLAM/Nav2/RViz
Pi	leo (LeoOS 2.3.0)	10.0.0.1	Ubuntu 24.04.3 / kernel 6.8.0-1041-raspi, runs Leo firmware
DDS	CycloneDDS	/etc/cyclonedds.xml	ROS 2 LOCALHOST_ONLY=0 — multi-machine discovery enabled

Workspaces

```
/home/team7/leo_test/                                # Main workspace (non-standard layout)
■■■■ package.xml                                       # leo_test ament_cmake package at ROOT
■■■■ CMakeLists.txt
■■■■ launch/real_robot.launch.py                       # Main launch file
■■■■ config/slam_params_real.yaml
■■■■ config/nav2_params_real.yaml
■■■■ scripts/odom_to_tf.py                             # NOT NEEDED — Pi already publishes odom TF
■■■■ src/                                              # Was polluted with rplidar_ros, since cleaned

/home/team7/ros2_ws/                                   # Separate workspace
■■■■ install/rplidar_ros/share/rplidar_ros/launch/
■■■■ view_rplidar_a2m12_launch.py                     # WORKING A2M12 launch (only here)
```

Required .bashrc Source Order (CRITICAL)

```
source /opt/ros/jazzy/setup.bash
source /home/team7/ros2_ws/install/setup.bash          # rplidar_ros workspace FIRST
source /home/team7/leo_test/install/setup.bash         # leo_test workspace SECOND
```

Order matters: ros2_ws contains the source-built rplidar_ros with the A2M12 launch file. The apt version of ros-jazzy-rplidar-ros lacks this launch and was previously removed.

2. The Single Unresolved Problem

All other issues in this 5-hour debugging session have been resolved. One issue remains:

NOTE: The map → odom TF transform is never published.

Consequences:

- `ros2 run tf2_ros tf2_echo map odom` reports *"Invalid frame ID 'map' passed to canTransform argument target_frame — frame does not exist"*
- RViz shows "No map received" when Fixed Frame is set to map
- Nav2 cannot plan paths because there is no global frame for goal poses
- SLAM Toolbox logs repeat: Message Filter dropping message: frame 'laser' for reason 'discarding message because the queue is full'

What This Pattern Suggests

- `slam_toolbox` is in lifecycle state `active` [3] (or sometimes stuck at `unconfigured` [1] depending on launch path)
- `/map` topic exists with one publisher
- But the `tf` broadcast of `map → odom` never appears
- Symptom is consistent with: (a) RViz QoS mismatch hiding a working map, (b) costmap inflation putting the rover in lethal cost (so Nav2 won't engage), or (c) SLAM still confused by some residual TF/timing issue

3. Confirmed Working Subsystems

These have been verified by direct terminal observation. Claude Code should treat these as **working baselines** and not modify them:

LiDAR

OK:
/scan publishes at ~12 Hz
RPLidar A2M12, /dev/ttyUSB0, baudrate 256000, scan_mode Sensitivity
User confirmed: 'using only lidar I can see everything, even if someone walks nearby it detects and shows scans moving'

Pi-side TF Publishers

Pi publishes odom → base_footprint correctly

```
team7@team7-NUC12WSKi7:~/leo_test$ ros2 topic echo /tf --once 2>/dev/null | grep -A 2 "frame_id" | head -20
    frame_id: odom
  child_frame_id: base_footprint
    transform:
      translation:
```

Pi-side Node Ecosystem (after reboot to clear duplicates)

ros2 node list – clean state, single instances of every node

```
/firmware
/firmware_message_converter
/firmware_parameter_bridge
/image_container
/imu_filter
/launch_ros_596
/leo_system
/odom_filter
/robot_state_publisher
/rosapi
/rosbridge_server
/web_video_server
```

Clock Sync

OK:
NUC: Fri Apr 24 12:35:49 PM UTC 2026
Pi: Fri Apr 24 12:35:49 UTC 2026
(within 4 seconds – well under TF tolerance)

4. Current Configuration Files (Relevant Excerpts)

/home/team7/leo_test/config/slam_params_real.yaml

```
slam_toolbox:
  ros__parameters:
    use_sim_time: false

    # Frame setup
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint    # Pi publishes base_footprint -> base_link via robot_state_
publisher
    scan_topic: /scan

    # Mode
    mode: mapping

    # SLAM tuning
    resolution: 0.05
    # ... (see file on disk for complete contents)
```

/home/team7/leo_test/config/nav2_params_real.yaml — Relevant Section

```
controller_server:
  ros__parameters:
    use_sim_time: false
    controller_plugins: ["FollowPath"]
    FollowPath:
      plugin: "nav2_regulated_pursuit_controller::RegulatedPurePursuitController"

local_costmap:
  local_costmap:
    ros__parameters:
      robot_radius: 0.25          # <-- Likely T00 BIG for tight lab
      inflation_layer:
        inflation_radius: 0.55    # <-- 0.25 + 0.55 = 0.8m clear zone needed

behavior_server:
  ros__parameters:
    recovery_plugins: ["spin", "backup", "wait"]
    spin:
      plugin: "nav2_behaviors::Spin"      # Jazzy double-colon syntax
    backup:
      plugin: "nav2_behaviors::BackUp"
    wait:
      plugin: "nav2_behaviors::Wait"
```

/home/team7/leo_test/launch/real_robot.launch.py — Current Structure

```
# Uses slam_toolbox's official online_async_launch.py
# (was previously a complex OnStateTransition chain – replaced)

lidar = IncludeLaunchDescription(
  PythonLaunchDescriptionSource(
    os.path.join(get_package_share_directory('rplidar_ros'),
                  'launch', 'rplidar_a2m12_launch.py')
  )
)

laser_tf = Node( # base_link -> laser static TF
  package='tf2_ros',
  executable='static_transform_publisher',
  arguments=[
    '--x', '0.0', '--y', '0.0', '--z', '0.25',
    '--roll', '0', '--pitch', '0', '--yaw', '0',
    '--frame-id', 'base_link', '--child-frame-id', 'laser'
  ]
)
```

```
slam = IncludeLaunchDescription( # slam_toolbox official launch
    PythonLaunchDescriptionSource(
        os.path.join(pkg_slam, 'launch', 'online_async_launch.py')
    ),
    launch_arguments={
        'use_sim_time': 'false',
        'slam_params_file': slam_config,
    }.items()
)

nav2 = TimerAction(
    period=15.0, # Delay Nav2 by 15s after launch
    actions=[IncludeLaunchDescription(...nav2_bringup/navigation_launch.py...)]
)
```

5. Already-Applied Fixes — DO NOT REPEAT

Claude Code: every item below has been applied and verified. Re-applying them will waste effort and possibly regress the setup.

Fix	What Was Done
Workspace overlay	Sourced <code>/home/team7/ros2_ws/install/setup.bash</code> before <code>leo_test</code>
A2M12 launch availability	<code>rplidar_ros</code> source-built into <code>ros2_ws</code> ; <code>apt rplidar_ros</code> removed
Nav2 plugin syntax	All Humble single-slash names converted to Jazzy double-colon (<code>nav2_behaviors::Spin</code> , <code>nav2_navfn_planner</code>)
progress_checker key	Renamed singular <code>progress_checker_plugin</code> → <code>progress_checker_plugins</code> (array)
SLAM YAML duplicates	Removed duplicate keys; <code>base_frame</code> set to <code>base_footprint</code> ; <code>use_lifecycle_manager</code> line removed entirely
Pi clock sync	Manually synced via <code>ssh pi@10.0.0.1 "sudo date -s '...'"</code> — within 4s
Pi duplicate processes	Forced full Pi reboot; node list confirmed single instances
USB permissions	<code>sudo chmod 666 /dev/ttyUSB0</code> before launch
Conflicting YAMLs	Removed <code>bt_nav_jazzy.yaml</code> , <code>controller_jazzy.yaml</code> , <code>planner_jazzy.yaml</code>
Launch complexity	Replaced <code>OnStateTransition</code> chain with <code>slam_toolbox</code> 's <code>online_async_launch.py</code>
Clean rebuild	<code>rm -rf build install log && colcon build</code> between fixes

NOTE: Despite all of the above, `map` → `odom` still does not publish. The remaining root cause is therefore in the live runtime behaviour — not in the configurations or installations themselves.

6. Verbatim Terminal Outputs From Latest Session

Claude Code: these are the actual outputs from the user's terminal *after* all fixes in Section 5 were applied. Use these to understand current runtime behaviour.

6.1 Clock Verification (Both Hosts)

```
team7@team7-NUC12WSKi7:~$ date -u
Fri Apr 24 12:31:11 PM UTC 2026

pi@leo:~$ date -u
Fri Apr 24 12:31:04 UTC 2026
```

6.2 SLAM Toolbox Lifecycle State

```
team7@team7-NUC12WSKi7:~/leo_test$ ros2 lifecycle get /slam_toolbox
unconfigured [1]
```

Issue: SLAM stuck in unconfigured state. Auto-configure mechanism not firing reliably. (After reboot of Pi and full restart, sometimes shows active [3] but TF still doesn't publish.)

6.3 /map Topic Status

```
team7@team7-NUC12WSKi7:~/leo_test$ ros2 topic hz /map
WARNING: topic [/map] does not appear to be published yet
```

Confirms /map not publishing when SLAM is unconfigured.

6.4 Persistent SLAM Log Pattern

```
ERROR:
[async_slam_toolbox_node-4] [INFO] [1777034064.632194498] [slam_toolbox]:
  Message Filter dropping message: frame 'laser' at time 1777034064.479
  for reason 'discarding message because the queue is full'
[async_slam_toolbox_node-4] [INFO] [1777034067.199271941] [slam_toolbox]:
  Message Filter dropping message: frame 'laser' at time 1777034067.046
  for reason 'discarding message because the queue is full'
[async_slam_toolbox_node-4] [INFO] [1777034059.504333728] [slam_toolbox]:
  Message Filter dropping message: frame 'laser' at time 1777034059.345
  for reason 'discarding message because the queue is full'
```

Pattern: SLAM receives scans but the message filter cannot match them to a TF transform within the queue lifetime. Even though clocks are synced, the queue still fills and drops messages.

6.5 tf2_echo map odom (current behaviour)

```
ERROR:
team7@team7-NUC12WSKi7:~/leo_test$ ros2 run tf2_ros tf2_echo map odom
[INFO] [tf2_echo]: Waiting for transform map -> odom: Invalid frame ID
"map" passed to canTransform argument target_frame -
frame does not exist
[INFO] [tf2_echo]: Waiting for transform map -> odom: Invalid frame ID
"map" passed to canTransform argument target_frame -
frame does not exist
```

6.6 Type Hash Warnings (Pi-side, Persistent)

```
[WARN] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic
'rt/firmware/battery' with type 'std_msgs::msg::dds_::Float32_'
from USER_DATA '(null)'.
[WARN] [rmw_cyclonedds_cpp]: Failed to parse type hash for topic
'rt/firmware/wheel_states' with type 'leo_msgs::msg::dds_::WheelStates_'
from USER_DATA '(null)'.
[... many more warnings of this type ...]
```

These are **cosmetic** — Pi uses micro-ROS DDS profiles that don't include type hashes. Cause no functional impact. Ignore.

6.7 Node List After Pi Reboot (Clean State)

```
OK:  
/firmware  
/firmware_message_converter  
/firmware_parameter_bridge  
/image_container  
/imu_filter  
/launch_ros_596  
/leo_system  
/odom_filter  
/robot_state_publisher  
/rosapi  
/rosbridge_server  
/web_video_server
```

Each node listed once (not duplicated). After the user runs `real_robot.launch.py`, NUC-side nodes (`/slam_toolbox`, `/controller_server`, etc.) join this list.

7. SLAM Toolbox Behaviour Summary

Observed Modes

SLAM Toolbox has been observed in three states across launches:

Lifecycle State	Meaning	Observed Symptoms
unconfigured [1]	Auto-configure event from launch file did not fire; publishers empty; queue-full warnings appear immediately	TF not published; /map empty
active [3]	Configure + Activate transitions completed	TF still not published; /map shows 1 publisher but no data; queue-full v
configuring → activating	Briefly seen during startup	Transient — usually progresses to one of the above within 5 seconds

The 'queue is full' Diagnostic

This message comes from `tf2_ros::MessageFilter` inside `slam_toolbox`. The flow is:

- LiDAR scan arrives on `/scan` topic
- `MessageFilter` tries to look up `scan_frame` → `base_frame` TF at the scan's timestamp
- Lookup fails or times out (queue keeps growing)
- Once queue exceeds capacity, oldest scans are dropped with this log message

What This Implies

The TF chain available to SLAM is:

```
odom          (Pi /odom_filter, 197 Hz)
  ■■■ base_footprint
    ■■■ base_link (Pi robot_state_publisher)
      ■■■ wheel_FL, wheel_FR, wheel_RL, wheel_RR (Pi)
      ■■■ laser (NUC static_transform_publisher in launch file)
```

Since SLAM expects `scan.frame_id` (`= 'laser'`) → `base_footprint` to be resolvable, and that path exists, the lookup *should* succeed. Yet the queue fills. This is the central mystery.

Possible Subtle Causes

- `scan.header.stamp` may be ahead of the latest TF timestamp in the buffer (TF transform timestamps lag the scan)
- `transform_timeout` in `slam_toolbox` may be too short for the cross-machine TF latency (default 0.2s; try increasing to 1.0s)
- `tf_buffer_duration` may not extend far enough back to cover slow TF arrivals
- Race condition: SLAM may have started before `robot_state_publisher` had broadcast all static transforms

8. Probability Table for Remaining Root Cause

Friends with similar setups suggested these possibilities, ranked by likelihood given the symptoms:

Hypothesis	Probability	Mechanism
Costmap inflation too aggressive	30%	robot_radius (0.25) + inflation_radius (0.55) = 0.8m clear zone needed. In tight lab, rover is in l
use_sim_time mismatch on a	25%	If any node has use_sim_time=true while others have false, TF buffer comparisons fail silently.
RViz QoS subscription mismatch	20%	/map publishes with Transient Local durability. RViz default for Map display is Volatile. RViz ne
SLAM transform_timeout too s	15%	Default transform_timeout in slam_toolbox (~0.2s) may be too short for cross-machine TF later
Other (tf_buffer_duration, scan timestamp diff)	10%	Less likely; would manifest differently.

9. Specific Tasks for Claude Code

Task 1 — Add transform_timeout to slam_params_real.yaml

File: /home/team7/leo_test/config/slam_params_real.yaml

Add or modify these parameters under slam_toolbox.ros__parameters:

```
transform_timeout: 1.0          # was default ~0.2s
tf_buffer_duration: 30.0        # cover slow TF arrivals
minimum_time_interval: 0.5      # don't oversample scans
stack_size_to_use: 40000000     # ensure enough memory for queue
transform_publish_period: 0.05  # publish map→odom at 20 Hz
```

Task 2 — Reduce Nav2 Costmap Inflation

File: /home/team7/leo_test/config/nav2_params_real.yaml

Apply these substitutions in BOTH local_costmap and global_costmap sections:

```
robot_radius: 0.20              # was 0.25
inflation_radius: 0.25          # was 0.55
cost_scaling_factor: 3.0        # leave as-is
```

Task 3 — Verify use_sim_time Across All Nodes

File: /home/team7/leo_test/config/nav2_params_real.yaml

For EVERY top-level node entry, ensure use_sim_time: false is present. The list of nodes to check:

- controller_server
- planner_server
- behavior_server
- bt_navigator
- waypoint_follower
- local_costmap.local_costmap
- global_costmap.global_costmap
- velocity_smoother
- collision_monitor
- smoother_server

Task 4 — Provide Default RViz Config With Correct QoS

Create file: /home/team7/leo_test/config/nav.rviz — preconfigured with Map display QoS Reliability=Reliable and Durability=TransientLocal. Update real_robot.launch.py to launch RViz with this config:

```
rviz = Node(
    package='rviz2',
    executable='rviz2',
    arguments=['-d', os.path.join(pkg_dir, 'config', 'nav.rviz')],
    parameters=[{'use_sim_time': False}]
)
```

Task 5 — Implement Plan C Fallback as a Separate Launch File

Create file: /home/team7/leo_test/launch/fallback_static_map.launch.py — runs LiDAR + base_link→laser TF + static map→odom TF + Nav2. Used when SLAM cannot be made to work in the lab session. Provides bulletproof navigation demo using only odometry + local costmap obstacle avoidance.

```
static_map_tf = Node(
    package='tf2_ros',
    executable='static_transform_publisher',
    name='fake_map_to_odom',
    arguments=['--x', '0', '--y', '0', '--z', '0',
               '--roll', '0', '--pitch', '0', '--yaw', '0',
               '--frame-id', 'map', '--child-frame-id', 'odom']
)
```

```
)
```

Task 6 — Build a rover_slam Package as Plan B

Create directory: /home/team7/leo_test/src/rover_slam/ — based on the working RSDPTeam10 batchmate config. Cartographer-based, no dependence on Pi odom. Reference: the setup_rover_slam.sh script in this conversation contains the full file tree and contents.

Run command (after build):

```
ros2 launch rover_slam slam_launch.py launch_rplidar:=true use_sim_time:=false
```

Task 7 — Add a Diagnostic Make Target / Helper Script

Create file: /home/team7/leo_test/scripts/diagnose.sh — runs a sequence of diagnostic commands and prints concise pass/fail per check. The set of checks should include:

- Clock differential between NUC and Pi (must be < 1s)
- /scan publish rate (must be > 5 Hz)
- /odom publish rate (must be > 20 Hz)
- Each TF link in the chain odom → base_footprint → base_link → laser
- map → odom TF (the failing one)
- use_sim_time on every Nav2 + SLAM node (must all be False)
- Lifecycle state of /slam_toolbox (must be active [3])
- /map publisher node name (must be /slam_toolbox)

10. Reference: Diagnostic Commands & Expected Outputs

Verify Workspace Sourcing

```
echo $AMENT_PREFIX_PATH | tr ':' '\n'

# Expected to include (in this order):
/home/team7/leo_test/install/leo_test
/home/team7/ros2_ws/install/rplidar_ros
/opt/ros/jazzy
```

Verify A2M12 Launch Available

```
find / -name "rplidar_a2m12_launch.py" 2>/dev/null

# Expected:
/home/team7/ros2_ws/install/rplidar_ros/share/rplidar_ros/launch/rplidar_a2m12_launch.py
/home/team7/ros2_ws/src/rplidar_ros/launch/rplidar_a2m12_launch.py
```

Pre-Launch Cleanup Sequence

```
ssh pi@10.0.0.1 "sudo date -s '$(date -u +%Y-%m-%d %H:%M:%S)'"
kill -9 -f ros2 ; pkill -9 -f slam ; pkill -9 -f nav2
pkill -9 -f rplidar ; pkill -9 -f cartographer ; sleep 3
sudo chmod 666 /dev/ttyUSB0
```

Post-Launch Verification (after 30s)

```
# Lifecycle state – must be active [3]
ros2 lifecycle get /slam_toolbox

# /map must have a publisher named /slam_toolbox
ros2 topic info /map --verbose

# /map must publish data
ros2 topic hz /map

# THE KEY TEST – must show Translation/Rotation
ros2 run tf2_ros tf2_echo map odom

# All must be False
for n in /slam_toolbox /controller_server /planner_server /bt_navigator /behavior_server; do
    echo "$n: $(ros2 param get $n use_sim_time 2>/dev/null)"
done
```

Generate TF Tree PDF for Visual Inspection

```
ros2 run tf2_tools view_frames -o /tmp/frames
# Outputs /tmp/frames.pdf and /tmp/frames.gv
```

Force Lifecycle Configure + Activate (if auto-fire fails)

```
ros2 lifecycle set /slam_toolbox configure
sleep 2
ros2 lifecycle set /slam_toolbox activate
```

End of technical brief for Claude Code.